

ECOLE NORMALE SUPÉRIEURE DE L'ENSEIGNEMENT
TECHNIQUE DE MOHAMMEDIA
UNIVERSITÉ HASSAN II DE CASABLANCA

École Normale Supérieure de
l'Enseignement Technique

Department of Information and Mathematics

IT Engineering, Cybersecurity and Digital Trust engineering

IT Engineering, Big Data and Cloud Computing

Module : Génie Logiciel

Compte rendu – DevOps : Intégration et Déploiement Continus (CI/CD)

Élèves :
Abderrahmane HEDDAYA
Mohamed Zouhair ROUISSIYA
Solaymân OUMOUOU
Yasser NAMEZ

Enseignant :
Mme. Souad AHRIZ

9 janvier 2026

Table des matières

1 Introduction **2**

2 DevOps : Concepts Clés **2**
2.1 Définition du DevOps 2
2.2 Pourquoi adopter le DevOps ? 2
2.3 Cycle de vie DevOps 3

3 Le Paradigme du Continu **3**

4 Pipeline CI/CD **4**
4.1 Présentation générale 4
4.2 Étapes du pipeline CI/CD 4

5 Intégration Continue (CI) **4**
5.1 Définition et principes 4
5.2 Principes fondamentaux de la CI 5
5.3 Importance de la CI 5
5.4 Outils d'Intégration Continue 5

6 Déploiement Continue (CD) **6**
6.1 Définition 6
6.2 Continuous Delivery vs Continuous Deployment 6
6.3 Processus de déploiement 6

7 Stratégies de Déploiement **7**
7.1 Déploiement Blue-Green 7
7.2 Déploiement Canary 7

8 Avantages du CI/CD en Génie Logiciel **8**

9 Limites et Défis **9**

10 Conclusion **10**

1 Introduction

Dans le contexte actuel du développement logiciel, les entreprises font face à des défis majeurs : la nécessité de livrer des fonctionnalités rapidement, d'assurer la qualité des produits, et de maintenir une collaboration efficace entre les équipes de développement et d'exploitation. C'est dans ce contexte que le DevOps, et plus particulièrement les pratiques d'Intégration Continue (CI) et de Déploiement Continue (CD), émergent comme des solutions incontournables.

Ce rapport présente les concepts fondamentaux du DevOps, explore en détail les pipelines CI/CD, et analyse les avantages ainsi que les défis liés à l'adoption de ces pratiques dans le développement logiciel moderne.

2 DevOps : Concepts Clés

2.1 Définition du DevOps

DevOps

DevOps est une contraction de **Development** (Développement) et **Operations** (Opérations). Il s'agit d'une philosophie et d'un ensemble de pratiques qui visent à unifier le développement logiciel et l'exploitation des systèmes informatiques.

Le DevOps repose sur plusieurs piliers fondamentaux :

- **Livraison continue** : permettre une mise en production rapide et fréquente des nouvelles fonctionnalités
- **Collaboration** : briser les silos entre les équipes de développement, de test et d'exploitation
- **Automatisation** : réduire les tâches manuelles répétitives et sources d'erreurs
- **Culture du continu** : instaurer une amélioration permanente à tous les niveaux du cycle de développement
- **Changement de paradigme** : transformer la manière dont les organisations conçoivent et livrent des logiciels

2.2 Pourquoi adopter le DevOps ?

L'adoption du DevOps répond à plusieurs objectifs stratégiques pour les organisations :

Maximisation de la productivité L'unification des équipes de développement et d'opérations permet d'éliminer les frictions et d'accélérer les processus de livraison.

Simplification du cycle de vie Le cycle de production logicielle devient plus fluide, avec des transitions automatisées entre les différentes phases.

Programmabilité et dynamisme Chaque phase du développement devient programmable, permettant une adaptation rapide aux changements.

Amélioration de la fréquence des livraisons Les déploiements peuvent être effectués plusieurs fois par jour au lieu de quelques fois par an.

Optimisation de la prévisibilité et de la sécurité Les processus automatisés réduisent les erreurs humaines et améliorent la fiabilité des déploiements.

Détection précoce des bugs L'intégration et les tests fréquents permettent d'identifier et de corriger les problèmes rapidement.

2.3 Cycle de vie DevOps

Le cycle de vie DevOps s'articule autour de huit phases principales qui forment une boucle continue :

1. **Plan** : planification des fonctionnalités et des sprints
2. **Code** : développement du code source
3. **Build** : compilation et construction des artefacts
4. **Test** : exécution des tests automatisés
5. **Release** : préparation de la version pour le déploiement
6. **Deploy** : déploiement dans les environnements cibles
7. **Operate** : exploitation et maintenance en production
8. **Monitor** : surveillance et collecte de métriques

Ce cycle intègre les pratiques agiles pour le développement, l'Intégration Continue (CI) pour les phases de build et test, et le Déploiement Continue (CD) pour les phases de release et deploy.

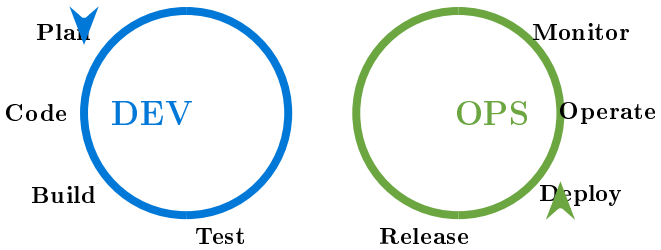


FIGURE 1 – Cycle de vie DevOps – Boucle infinie d'amélioration continue

3 Le Paradigme du Continu

Le DevOps introduit la notion de continuité à chaque étape du cycle de vie du développement logiciel (SDLC). Cette approche se décline en plusieurs dimensions :

— **Développement Continue** : intégration régulière du code des développeurs
— **Intégration Continue** : fusion et validation automatique du code
— **Test Continue** : exécution automatique des tests à chaque modification
— **Livraison Continue** : préparation automatique des versions déployables
— **Déploiement Continue** : mise en production automatique
— **Surveillance Continue** : monitoring permanent des applications en production

4 Pipeline CI/CD

4.1 Présentation générale

Un pipeline CI/CD est une chaîne automatisée qui transforme le code source en une application déployée en production. Il se compose de deux parties principales : le pipeline d'Intégration Continue (CI) et le pipeline de Déploiement Continue (CD).

4.2 Étapes du pipeline CI/CD

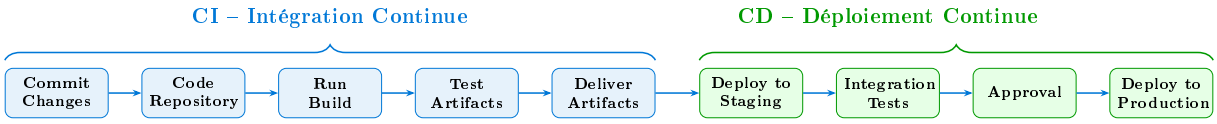


FIGURE 2 – Architecture d'un pipeline CI/CD complet

Le pipeline complet comprend les étapes suivantes :

Phase CI (Intégration Continue)

1. **Commit Changes** : le développeur pousse ses modifications dans le dépôt
2. **Code Repository** : le système de gestion de versions stocke le code (Git, GitLab, GitHub)
3. **Run Build** : compilation du code et création des artefacts
4. **Test Artifacts** : exécution des tests unitaires et d'intégration
5. **Deliver Artifacts** : stockage des artefacts validés dans un registre (Container Registry, Artifact Registry)

Phase CD (Déploiement Continue)

1. **Deploy to Staging** : déploiement dans un environnement de pré-production
2. **Run Integration Tools** : tests d'intégration et de performance
3. **Approval** : validation (manuelle ou automatique) avant la production
4. **Deploy to Production** : déploiement dans l'environnement de production

5 Intégration Continue (CI)

5.1 Définition et principes

Intégration Continue

L'Intégration Continue est une pratique DevOps consistant à intégrer fréquemment le code dans un dépôt partagé. Chaque intégration déclenche automatiquement un build et des tests, permettant de détecter rapidement les problèmes.

5.2 Principes fondamentaux de la CI

Les principes essentiels de l'Intégration Continue incluent :

— **Commits fréquents** : les développeurs intègrent leur code plusieurs fois par jour
— **Dépôt central** : utilisation d'un système de gestion de versions comme Git
— **Build automatique** : chaque commit déclenche une compilation automatique
— **Tests automatisés** : exécution systématique de la suite de tests
— **Intégration rapide** : feedback rapide sur la qualité du code

5.3 Importance de la CI

L'Intégration Continue apporte plusieurs bénéfices majeurs :

— **Détection précoce des bugs** : les problèmes sont identifiés immédiatement après leur introduction
— **Réduction des conflits de code** : les intégrations fréquentes minimisent les conflits de merge
— **Amélioration de la qualité du logiciel** : les tests automatiques garantissent un niveau de qualité constant
— **Feedback rapide** : les développeurs reçoivent immédiatement un retour sur leurs modifications

5.4 Outils d'Intégration Continue

Plusieurs outils sont disponibles pour mettre en place l'Intégration Continue :

— **Jenkins** : plateforme open-source populaire et extensible
— **GitHub Actions** : intégré nativement à GitHub
— **GitLab CI** : solution intégrée à GitLab
— **Azure DevOps** : plateforme complète de Microsoft
— **SonarQube** : outil spécialisé dans l'analyse de la qualité du code

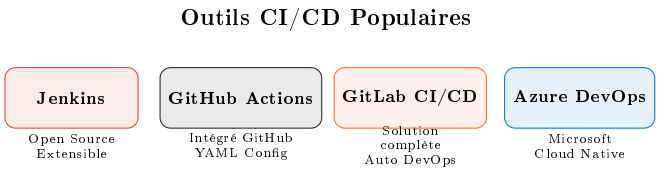


FIGURE 3 – Principaux outils d'Intégration Continue

6 Déploiement Continue (CD)

6.1 Définition

Déploiement Continue

Le Déploiement Continue est une extension de l'Intégration Continue qui automatise le déploiement du logiciel en production. Chaque changement validé passe automatiquement à travers le pipeline et est déployé en production sans intervention humaine.

6.2 Continuous Delivery vs Continuous Deployment

Il est important de distinguer deux concepts souvent confondus :

Continuous Delivery (Livraison Continue)

- Le processus est automatisé jusqu'à la phase de déploiement en production
- Une intervention humaine est nécessaire pour valider le déploiement final
- Séquence : Code → Build → Test → Approve → Release → **[Manuel]** → Deploy

Continuous Deployment (Déploiement Continue)

- Le déploiement en production est entièrement automatisé
- Aucune validation humaine n'est nécessaire
- Séquence : Code → Build → Test → Approve → Release → **[Auto]** → Deploy

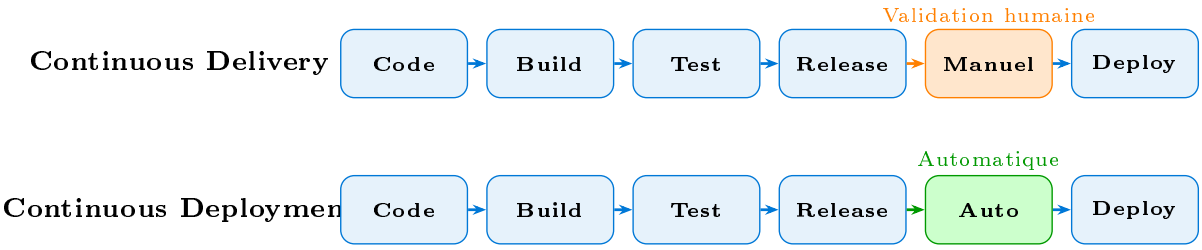


FIGURE 4 – Comparaison : Continuous Delivery vs Continuous Deployment

6.3 Processus de déploiement

Le processus de Déploiement Continue comprend plusieurs étapes :

1. Récupération des artefacts validés depuis le registre
2. Déploiement automatique dans l'environnement de staging
3. Exécution des tests d'intégration et de performance
4. Validation (automatique ou manuelle selon le mode choisi)
5. Déploiement en production selon la stratégie définie

7 Stratégies de Déploiement

7.1 Déploiement Blue-Green

Le déploiement Blue-Green utilise deux environnements de production identiques :

Principe de fonctionnement

- **Blue** : environnement de production actuel servant le trafic utilisateur
- **Green** : environnement où la nouvelle version est déployée
- La nouvelle version est testée sur Green pendant que Blue reste actif
- Basculement instantané du trafic de Blue vers Green après validation
- Blue devient l'environnement de secours en cas de problème

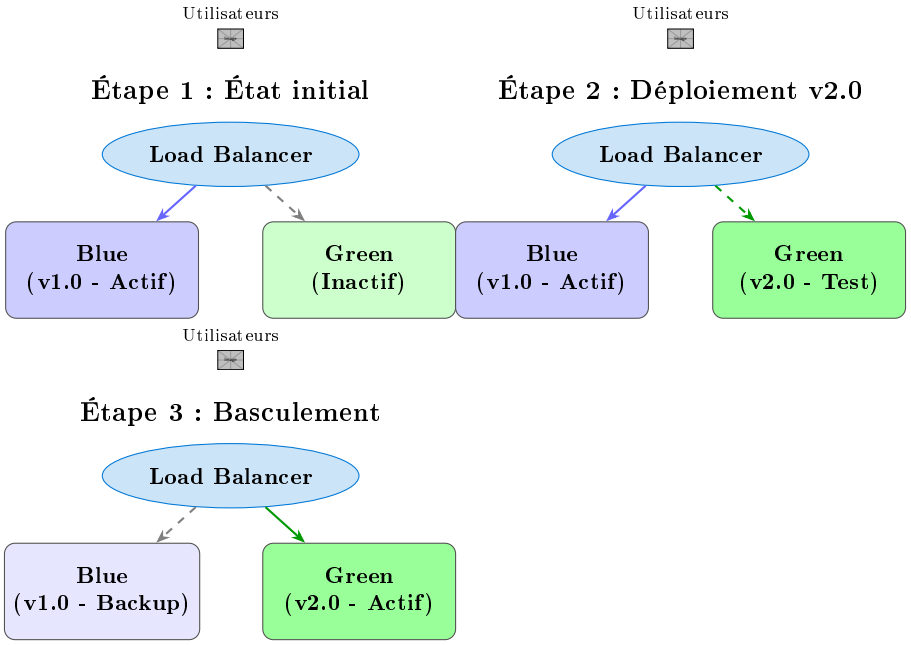


FIGURE 5 – Stratégie de déploiement Blue-Green

Avantages

- **Rollback rapide** : retour à la version précédente quasi-instantané
- **Zéro interruption** : pas de downtime pendant le déploiement
- **Tests en conditions réelles** : validation sur un environnement identique à la production

7.2 Déploiement Canary

Le déploiement Canary (en référence aux canaris utilisés dans les mines) introduit progressivement la nouvelle version :

Principe de fonctionnement

- La nouvelle version est déployée pour un petit pourcentage d'utilisateurs (ex : 5%)
- Surveillance étroite du comportement et des métriques d'erreur
- Augmentation progressive du trafic vers la nouvelle version (10%, 25%, 50%, 100%)
- Possibilité d'arrêter le déploiement à tout moment si des problèmes sont détectés

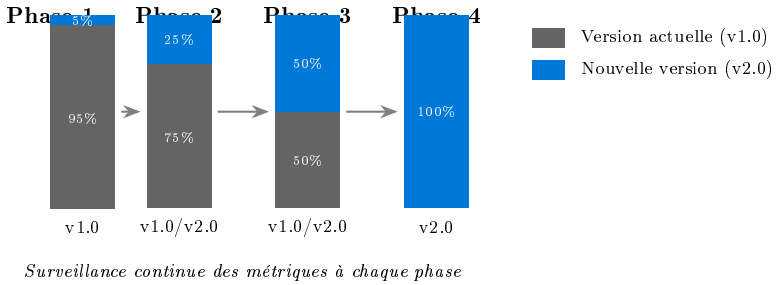


FIGURE 6 – Stratégie de déploiement Canary – Augmentation progressive du trafic

Avantages

- **Réduction des risques** : limitation de l'impact en cas de problème
- **Détection précoce** : identification rapide des problèmes sur un sous-ensemble d'utilisateurs
- **Validation progressive** : confiance croissante avant le déploiement complet

8 Avantages du CI/CD en Génie Logiciel

L'adoption des pratiques CI/CD apporte de nombreux bénéfices aux organisations :

Avantages

- **Détection précoce des bugs** : les problèmes sont identifiés et corrigés rapidement, réduisant le coût de correction
- **Amélioration de la qualité du logiciel** : les tests automatisés garantissent un niveau de qualité constant
- **Livraison rapide et fréquente** : réduction du time-to-market pour les nouvelles fonctionnalités
- **Réduction des erreurs humaines** : l'automatisation élimine les erreurs liées aux déploiements manuels
- **Automatisation des tests et des déploiements** : gain de temps considérable pour les équipes
- **Meilleure collaboration** : renforcement de la communication entre développeurs, testeurs et équipes opérationnelles
- **Feedback continu** : les développeurs reçoivent rapidement des retours sur la qualité de leur code
- **Réduction des risques** : les déploiements fréquents contiennent moins de changements, donc moins de risques

9 Limites et Défis

Malgré ses nombreux avantages, l'adoption du CI/CD présente également des défis :

Limites et Défis

- **Coût initial de mise en place** : investissement important en temps et en ressources pour configurer les pipelines
- **Complexité des pipelines CI/CD** : la maintenance et l'évolution des pipelines peuvent devenir complexes
- **Dépendance à la qualité des tests automatisés** : des tests insuffisants ou de mauvaise qualité réduisent l'efficacité du CI/CD
- **Courbe d'apprentissage** : les équipes doivent se former aux nouveaux outils et pratiques
- **Problèmes de sécurité** : gestion des secrets, des accès et des permissions dans les pipelines
- **Gestion des échecs en production** : nécessité de mettre en place des mécanismes de rollback efficaces
- **Changement culturel** : résistance au changement et nécessité d'adopter une nouvelle mentalité

10 Conclusion

Le DevOps, et plus particulièrement les pratiques d'Intégration Continue et de Déploiement Continue, représentent une évolution majeure dans le développement logiciel moderne. En unifiant les équipes de développement et d'exploitation, en automatisant les processus répétitifs et en instaurant une culture du continu, le DevOps permet aux organisations de livrer des logiciels de meilleure qualité, plus rapidement et de manière plus fiable.

Les pipelines CI/CD, en automatisant l'intégration, les tests et le déploiement, réduisent considérablement les risques liés aux mises en production et permettent un feedback rapide aux développeurs. Les stratégies de déploiement comme Blue-Green et Canary offrent des mécanismes sophistiqués pour minimiser l'impact des problèmes potentiels.

Cependant, l'adoption du DevOps nécessite un investissement significatif en termes de temps, de ressources et de formation. Le succès d'une transformation DevOps repose non seulement sur les outils et les technologies, mais surtout sur un changement culturel profond au sein des organisations.

Dans un contexte où la rapidité de livraison et la qualité des logiciels sont des avantages compétitifs majeurs, le DevOps et le CI/CD s'imposent comme des pratiques incontournables pour toute organisation souhaitant rester compétitive dans le domaine du développement logiciel.